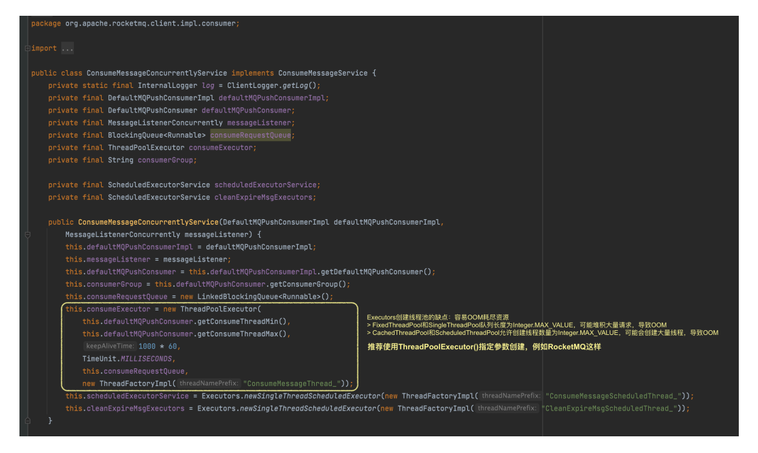

```java
package org.apache.rocketmq.client.impl.consumer;

import ...

public class ConsumeMessageConcurrentlyService implements ConsumeMessageService {
    private static final InternalLogger log = ClientLogger.getLog();
    private final DefaultMQPushConsumerImpl defaultMQPushConsumerImpl;
    private final DefaultMQPushConsumer defaultMQPushConsumer;
    private final MessageListenerConcurrently messageListener;
    private final BlockingQueue<Runnable> consumeRequestQueue;
    private final ThreadPoolExecutor consumeExecutor;
    private final String consumerGroup;

    private final ScheduledExecutorService scheduledExecutorService;
    private final ScheduledExecutorService cleanExpireMsgExecutors;

    public ConsumeMessageConcurrentlyService(DefaultMQPushConsumerImpl defaultMQPushConsumerImpl,
        MessageListenerConcurrently messageListener) {
        this.defaultMQPushConsumerImpl = defaultMQPushConsumerImpl;
        this.messageListener = messageListener;
        this.defaultMQPushConsumer = this.defaultMQPushConsumerImpl.getDefaultMQPushConsumer();
        this.consumerGroup = this.defaultMQPushConsumer.getConsumerGroup();
        this.consumeRequestQueue = new LinkedBlockingQueue<Runnable>();
        this.consumeExecutor = new ThreadPoolExecutor(
            this.defaultMQPushConsumer.getConsumeThreadMin(),
            this.defaultMQPushConsumer.getConsumeThreadMax(),
             keepAliveTime: 1000 * 60,
            TimeUnit.MILLISECONDS,
            this.consumeRequestQueue,
            new ThreadFactoryImpl( threadNamePrefix: "ConsumeMessageThread_"));
        this.scheduledExecutorService = Executors.newSingleThreadScheduledExecutor(new ThreadFactoryImpl( threadNamePrefix: "ConsumeMessageScheduledThread_"));
        this.cleanExpireMsgExecutors = Executors.newSingleThreadScheduledExecutor(new ThreadFactoryImpl( threadNamePrefix: "CleanExpireMsgScheduledThread_"));
    }
```

Executors创建线程池的缺点：容易OOM耗尽资源
> FixedThreadPool和SingleThreadPool队列长度为Integer.MAX_VALUE，可能堆积大量请求，导致OOM
> CachedThreadPool和ScheduledThreadPool允许创建线程数量为Integer.MAX_VALUE，可能会创建大量线程，导致OOM

推荐使用ThreadPoolExecutor()指定参数创建，例如RocketMQ这样

```java
java.util.concurrent.Executors.newFixedThreadPool(int nThreads){
  return new ThreadPoolExecutor(nThreads, nThreads, 0L, TimeUnit.MILLISECONDS, new LinkedBlockingQueue<Runnable>()); //LinkedBlockingQueue的容量为Integer.MAX_VALUE约21亿
}                                                                                                  //这么大的队列容量，将会影响线程池的执行逻辑，详见下方
```

```java
java.util.concurrent.ThreadPoolExecutor.ThreadPoolExecutor(int corePoolSize,
                                                           int maximumPoolSize,
                                                           long keepAliveTime,
                                                           TimeUnit unit,
                                                           BlockingQueue<Runnable> workQueue) {
  this(corePoolSize, maximumPoolSize, keepAliveTime, unit, workQueue, Executors.defaultThreadFactory(), defaultHandler);
}

java.util.concurrent.ThreadPoolExecutor.ThreadPoolExecutor(int corePoolSize,
                                                           int maximumPoolSize,
                                                           long keepAliveTime,
                                                           TimeUnit unit,
                                                           BlockingQueue<Runnable> workQueue
                                                           ThreadFactory threadFactory,
                                                           RejectedExecutionHandler handler) {
  if (corePoolSize < 0 || maximumPoolSize ≤ 0 || maximumPoolSize < corePoolSize || keepAliveTime < 0) throw new IllegalArgumentException();
  if (workQueue == null || threadFactory == null || handler == null) throw new NullPointerException();
  this.acc = System.getSecurityManager() == null ? null : AccessController.getContext();
  this.corePoolSize = corePoolSize; //核心线程数
  this.maximumPoolSize = maximumPoolSize; //最大线程数
  this.workQueue = workQueue; //暂存task的队列
  this.keepAliveTime = unit.toNanos(keepAliveTime); //线程数>corePoolSize时，多余空闲线程等待任务的最长时间，超过这个时间后，多余线程会被终止
  this.threadFactory = threadFactory;
  this.handler = handler;
}
```

```java
public void ThreadPoolExecutor.execute(Runnable command) {
  int c = ctl.get();
  if (workerCountOf(c) < corePoolSize) {
    //1. 当前线程数小于核心线程数，启动新核心线程
    if (addWorker(command, true)) return;
    c = ctl.get();
  }
  //2. 线程池完成预热（所有核心线程启动完成）处于RUNNING状态，任务先加入阻塞队列
  if (isRunning(c) && workQueue.offer(command)) {
    int recheck = ctl.get();
    //线程池不再是RUNNING状态，从阻塞队列中删除任务，并抛出拒绝异常
    if (! isRunning(recheck) && remove(command)) reject(command);
    //如果这时线程池为空，启动新线程
    else if (workerCountOf(recheck) == 0) addWorker(null, false);
    //3. 队列满了，尝试直接新增线程，若失败抛出异常（因为阻塞队列是无界队列，这段逻辑基本不会执行）
  } else if (!addWorker(command, false)) reject(command);
}
```

```java
void reject(Runnable command) → handler.rejectedExecution(command, this);

AbortPolicy.rejectedExecution(Runnable r, ThreadPoolExecutor e) { //默认策略
  throw new RejectedExecutionException("Task " + r.toString() + " rejected from " + e.toString());
}

CallerRunsPolicy.rejectedExecution(Runnable r, ThreadPoolExecutor e) {
  if (!e.isShutdown()) { r.run(); } //在caller线程里，直接执行
}

DiscardPolicy.rejectedExecution(Runnable r, ThreadPoolExecutor e) { /* do nothing */ }

DiscardOldestPolicy.rejectedExecution(Runnable r, ThreadPoolExecutor e) {
  if (!e.isShutdown()) {
    e.getQueue().poll(); //取出queue里最早入队的task，直接丢弃
    e.execute(r); //重新提交
  }
}
```

```java
private boolean addWorker(Runnable firstTask, boolean core) {
  retry:
  for (;;) { //自旋，更新workers数量
    int c = ctl.get();
    int rs = runStateOf(c);
    //线程池已关闭 or workQueue为空，不再接受新任务，返回false
    if (rs ≥ SHUTDOWN && !(rs == SHUTDOWN && firstTask == null && ! workQueue.isEmpty())) return false;
    for (;;) {
      int wc = workerCountOf(c);
      //workers够了，不能再加，返回false
      if (wc ≥ CAPACITY || wc ≥ (core ? corePoolSize : maximumPoolSize)) return false;
      if (compareAndIncrementWorkerCount(c)) break retry; //workers数量加1，结束自旋
      c = ctl.get();
      if (runStateOf(c) ≠ rs) continue retry; //线程池状态变了，重新自旋判断
    }
  }
  boolean workerStarted = false;
  boolean workerAdded = false;
  Worker w = null;
  try {
    w = new Worker(firstTask); //1.新建worker
    final Thread t = w.thread;
    if (t ≠ null) {
      final ReentrantLock mainLock = this.mainLock;
      mainLock.lock(); //获取thread pool的独占锁，实现workers同步，同一时间只有一个线程操作workers
      try {
        int rs = runStateOf(ctl.get());
        if (rs < SHUTDOWN || (rs == SHUTDOWN && firstTask == null)) {
          if (t.isAlive()) throw new IllegalThreadStateException();
          workers.add(w);  //2.保存新线程
          int s = workers.size();
          if (s > largestPoolSize) largestPoolSize = s;
          workerAdded = true; //3.更新状态
        }
      } finally { mainLock.unlock(); }
      if (workerAdded) {
        t.start(); //3. 启动新线程
        workerStarted = true;
      }
    }
  } finally {
    if (!workerStarted) addWorkerFailed(w); //创建worker失败，回滚
  }
  return workerStarted;
}
```

```java
Worker extends AbstractQueuedSynchronizer implements Runnable {
  final Thread thread;
  Runnable firstTask;
  volatile long completedTasks;
  Worker(Runnable firstTask) {
    setState(-1); //worker.state设置为-1，避免task.run()之前被中断
    this.firstTask = firstTask;
    this.thread = getThreadFactory().newThread(this); //创建线程
  }
}
```

```java
/* 解耦线程的创建和使用，将繁琐的线程创建及信息配置封装起来，对外提供简单的接口 */
public Thread DefaultThreadFactory.newThread(Runnable r) {
  Thread t = new Thread(group, r, namePrefix + threadNumber.getAndIncrement(), 0);
   if (t.isDaemon()) t.setDaemon(false);
   if (t.getPriority() ≠ Thread.NORM_PRIORITY) t.setPriority(Thread.NORM_PRIORITY);
  return t;
}
```

```java
Thread.run() → Worker.run() → runWorker(this) {
  Thread wt = Thread.currentThread();
  Runnable task = w.firstTask;
  w.firstTask = null;
  w.unlock(); //worker.state设置为0，允许中断
  boolean completedAbruptly = true;
  try {
    while (task ≠ null || (task = getTask()) ≠ null) { //执行完当前任务后，反复从workQueue中获取任务执行
      w.lock(); //1. Worker加锁（state设置为1），防止正在执行的任务因其他线程调用shutdown()而中断
      if ((runStateAtLeast(ctl.get(), STOP) || (Thread.interrupted() && runStateAtLeast(ctl.get(), STOP))) && !wt.isInterrupted()) wt.interrupt();
      try {
        beforeExecute(wt, task);
        Throwable thrown = null;
        try {
          task.run(); //2. 执行用户定义的业务逻辑
        } catch (RuntimeException x) { thrown = x; throw x;}
          catch (Error x) { thrown = x; throw x; }
          catch (Throwable x) { thrown = x; throw new Error(x); }
          finally { afterExecute(task, thrown);  }
      } finally {
        task = null;
        w.completedTasks++; //3. worker执行的task数加1
        w.unlock(); //4. woker解锁
      }
    }
    completedAbruptly = false;
  } finally {
    processWorkerExit(w, completedAbruptly); //5. 关闭worker
  }
}
```

```java
private Runnable getTask() {
  boolean timedOut = false; //Did the last poll() time out?
  for (;;) { //自旋，不断地尝试从workQueue中取task
    int c = ctl.get();
    int rs = runStateOf(c);
    if (rs ≥ SHUTDOWN && (rs ≥ STOP || workQueue.isEmpty())) {
      decrementWorkerCount();
      return null; //1. 线程池已经停止，直接返回
    }
    int wc = workerCountOf(c);
    boolean timed = allowCoreThreadTimeOut || wc > corePoolSize;
    if ((wc > maximumPoolSize || (timed && timedOut)) && (wc > 1 || workQueue.isEmpty())) {
      if (compareAndDecrementWorkerCount(c)) return null; //2. 当前线程数超过最大线程数，或者等待超时，直接返回
      continue;
    }

    try {
      Runnable r = timed ? workQueue.poll(keepAliveTime, TimeUnit.NANOSECONDS) : workQueue.take();
      if (r ≠ null) return r; //3. 从队列中取到任务，返回
      timedOut = true; //超时了，继续循环
    } catch (InterruptedException retry) {
      timedOut = false;
    }
  }
}
```

```java
//Performs cleanup and bookkeeping for a dying worker.
void processWorkerExit(Worker w, boolean completedAbruptly) {
  if (completedAbruptly) decrementWorkerCount(); //ctl - 1
  final ReentrantLock mainLock = this.mainLock;
  mainLock.lock();
  try {
    completedTaskCount += w.completedTasks;
    workers.remove(w); //从workers中移除当前worker
  } finally { mainLock.unlock(); }

  tryTerminate();

  int c = ctl.get();
  if (runStateLessThan(c, STOP)) {
    if (!completedAbruptly) {
      int min = allowCoreThreadTimeOut ? 0 : corePoolSize;
      if (min == 0 && ! workQueue.isEmpty()) min = 1;
      if (workerCountOf(c) ≥ min) return;
    }
    addWorker(null, false); //worker异常结束时，新增1个
  }
}
```

```java
// Previously submitted tasks are executed, but no new tasks will be accepted.
// This method does not wait for previously submitted tasks to complete execution.
public void ThreadPoolExecutor.shutdown() {
  final ReentrantLock mainLock = this.mainLock;
  mainLock.lock();
  try {
    checkShutdownAccess(); //make sure caller has permission to shut down threads
    advanceRunState(SHUTDOWN); //设置状态
    interruptIdleWorkers();
    onShutdown();
  } finally {
    mainLock.unlock();
  }
  tryTerminate();
}
```

```java
//Interrupts threads that might be waiting for tasks
private void interruptIdleWorkers() → interruptIdleWorkers(false) {
  final ReentrantLock mainLock = this.mainLock;
  mainLock.lock(); //1. 获取全局锁，保证只有1个线程可以调用shutdown()
  try {
    for (Worker w : workers) {
      Thread t = w.thread;
      if (!t.isInterrupted() && w.tryLock()) { //2. 如果w正在执行任务，tryLock失败，直接跳过
        try {
          t.interrupt(); //3. 设置线程中断标志
        } catch (SecurityException ignore) {
        } finally { w.unlock(); } //4. 解除worker锁
      }
      if (onlyOne) break;
    }
  } finally {
    mainLock.unlock(); //解除全局锁
  }
}
```

```java
boolean Worker.tryLock() → tryAcquire(1) {
  if (compareAndSetState(0, 1)) {
    setExclusiveOwnerThread(Thread.currentThread());
    return true;
  }
  return false;
}
```

```java
//Transitions to TERMINATED state
final void tryTerminate() {
  for (;;) {
    int c = ctl.get();
    if (isRunning(c) || runStateAtLeast(c, TIDYING) || (runStateOf(c) == SHUTDOWN && ! workQueue.isEmpty())) return;
    if (workerCountOf(c) ≠ 0) {
      interruptIdleWorkers(ONLY_ONE);
      return;
    }
    final ReentrantLock mainLock = this.mainLock;
    mainLock.lock(); //1. 获取全局锁
    try {
      if (ctl.compareAndSet(c, ctlOf(TIDYING, 0))) { //2. 线程池状态设置为TIDYING
        try {
          terminated();
        } finally {
          ctl.set(ctlOf(TERMINATED, 0)); //3. 线程池状态设置为TERMINATED
          termination.signalAll(); //4. 通知awaitTermination()的线程
        }
      }
    } finally { mainLock.unlock(); } //5. 释放全局锁
  }
}
```

```java
final void tryTerminate() {
  for (;;) {

}
```

```java
// Previously submitted tasks are executed, but no new tasks will be accepted.
// This method does not wait for previously submitted tasks to complete execution.
public List<Runnable> ThreadPoolExecutor.shutdownNow() {
  List<Runnable> tasks;
  final ReentrantLock mainLock = this.mainLock;
  mainLock.lock();
  try {
    checkShutdownAccess(); //make sure caller has permission to shut down threads
    advanceRunState(STOP); //设置状态
    interruptWorkers(); //中断所有线程
    tasks = drainQueue(); //收集还未开始执行的任务
  } finally {
    mainLock.unlock();
  }
  tryTerminate();
  return tasks;
}
```

```java
//Interrupts all threads, even if active.
private void interruptWorkers() {
  final ReentrantLock mainLock = this.mainLock;
  mainLock.lock(); //1. 获取全局锁，保证只有1个线程可以调用shutdown()
  try {
    for (Worker w : workers) { //2. 中断所有线程
      w.interruptIfStarted();
    }
  } finally {
    mainLock.unlock(); //3. 解除全局锁
  }
}
```

```java
void Worker.interruptIfStarted() {
  Thread t;
  if (getState() ≥ 0 && (t = thread) ≠ null && !t.isInterrupted()) {
    try {
      t.interrupt();
    } catch (SecurityException ignore) { }
  }
}
```

```java
//Drains the task queue into a new list
private List<Runnable> drainQueue() {
  BlockingQueue<Runnable> q = workQueue;
  ArrayList<Runnable> taskList = new ArrayList<Runnable>();
  q.drainTo(taskList);
  if (!q.isEmpty()) { //如果drain失败了，一个一个处理
    for (Runnable r : q.toArray(new Runnable[0])) {
      if (q.remove(r)) taskList.add(r);
    }
  }
  return taskList;
}
```